

Nebula@BITS Goa — Project Abstract

Topic: Constraint Optimization through RTL Enhancement Using Generative AI (Digital track, Astera Labs)

Team size: up to 3 (students) **Abstract submission:** 6 August · **Final submission:** 15 September ·

Presentation: 25 September

Title

A Tool-Grounded, Human-Reviewable Framework for GenAI-Assisted RTL Timing Optimization with Formal Equivalence Gating

Abstract

Achieving timing closure remains one of the most iterative activities in digital design: engineers repeatedly read static-timing reports, trace critical paths back to RTL, hand-edit the source, and re-run synthesis to check whether a change actually helped without breaking behavior. Existing public work on large language models for RTL is dominated by small specification-to-RTL generation or generic power–performance–area (PPA) rewriting benchmarks, while commercial AI-in-EDA offerings largely expose closed, flow-level recipe optimization; neither adequately demonstrates a transparent, auditable loop that links *parsed timing paths* to *bounded RTL context*, records every candidate, and gates changes with *open, deterministic* timing and formal-equivalence evidence under a deliberately multi-clock, CDC-bearing benchmark.

We propose **Nebula**, a framework in which a Python orchestrator drives deterministic EDA tools and a generative-AI component proposes constrained RTL changes, so that a reviewer can trace the full chain — *constraints* → *failing timing path* → *mapped RTL context* → *proposed transformation* → *candidate RTL* → *measured QoR* → *equivalence result* → *accept/reject*. Authority is deliberately split: the LLM only summarizes a structured critical path and emits a schema-constrained, size-bounded candidate patch with its rationale, predicted trade-off, and assumptions, while deterministic tools alone parse reports, apply patches to an isolated candidate, measure results, and decide the verdict — the model can never set `optimization_succeeded`. Static timing analysis and synthesis use OpenSTA, Yosys, and OpenROAD (via OpenROAD Flow Scripts); functional preservation is checked by formal equivalence (EQY, with SymbiYosys for supporting property proofs) under an explicitly declared equivalence relation — cycle-exact for combinational and synthesis-friendly refactors first, with sequential/latency-aware relations reserved for pipelining and retiming — and any UNKNOWN or timeout result is never relabeled as a pass.

We will evaluate on a two-tier benchmark: a tiny "truth fixture" for rapid regression, and one qualified benchmark satisfying the official conditions — five independent master asynchronous clock domains, at least one generated clock per master, clock-domain crossings, multi-ratio divider logic, and approximately 50K standard cells. Baseline and candidate runs will be compared under frozen, identical settings (same toolchain, library, corner, and constraints), with ablations against manual and synthesis-only baselines and full retention of rejected and failed attempts to avoid cherry-picking. The expected contribution is the framework itself together with its versioned data schemas and an empirical account of where GenAI assistance is and is not useful under real correctness constraints — not a claim of autonomous or tapeout-ready timing signoff. If validated, the approach would reduce the manual read-report/edit/re-run cycle while keeping every accepted change auditable and formally verified, with the engineer in control.

(~360 words — trim to the organiser's limit using the annotated version below.)

Annotated version (maps each sentence to the required abstract framework)

#	Framework element	Sentence(s) used
1	Brief context — why RTL-level timing feedback matters	"Achieving timing closure remains one of the most iterative activities..."
2	Specific gap — what current work/tools do not expose	"Existing public work... neither adequately demonstrates a transparent, auditable loop..."
3	Proposed direction — name the framework and its boundary	"We propose Nebula , a framework in which a Python orchestrator drives deterministic EDA tools..."
4	AI + deterministic interaction — assign authority	"Authority is deliberately split: the LLM only... while deterministic tools alone... — the model can never set <code>optimization_succeeded</code> ."
5	Verification strategy — establish safety, relation, and failure handling	"...functional preservation is checked by formal equivalence (EQY...) under an explicitly declared equivalence relation... UNKNOWN or timeout... never relabeled as a pass."
6	Evaluation plan — make claims falsifiable	"We will evaluate on a two-tier benchmark... frozen, identical settings... ablations... retention of rejected and failed attempts."
7	Expected contribution — artifact/knowledge	"The expected contribution is the framework itself together with its versioned data schemas and an empirical account..."
8	Practical importance — user value	"If validated, the approach would reduce the manual... cycle while keeping every accepted change auditable and formally verified, with the engineer in control."

Short version (~150 words, if a tight limit applies)

Timing closure is iterative: engineers read timing reports, trace critical paths to RTL, hand-edit, and re-synthesize to check gains without breaking behavior. Public LLM-for-RTL work centers on small code-generation or generic PPA rewriting, and commercial AI-EDA tools expose closed flow-level tuning; a transparent loop linking *parsed timing paths* to *bounded RTL context* under *open, deterministic* verification on a multi-clock, CDC-bearing design is under-demonstrated. We propose **Nebula**: a Python orchestrator runs OpenSTA/Yosys/OpenROAD and formal equivalence (EQY/SymbiYosys), while a generative-AI component only proposes schema-constrained, size-bounded RTL patches with rationale and predicted trade-offs — deterministic tools measure QoR and decide accept/reject. We will evaluate on a tiny regression fixture plus one benchmark meeting the official five-asynchronous-clock, generated-clock, CDC, divider, and ~50K-cell conditions, comparing baseline and candidates under frozen settings with ablations and full retention of failed attempts. The contribution is an auditable, formally gated framework and its schemas — not autonomous timing signoff.

Keywords

timing closure · RTL optimization · static timing analysis · critical-path analysis · timing constraints · generative AI · tool-grounded feedback · formal equivalence checking · power-performance-area (PPA) · human-in-the-loop · clock-domain crossing · generated clocks · reproducible EDA · structured output · candidate verification

Terms deliberately avoided

autonomous signoff · guaranteed optimization · tapeout-ready · novel · industry-grade · state of

the art — none of these are supported by evidence at abstract stage, and the framing is kept in the future tense ("we propose", "we will evaluate", "expected contribution") with no fabricated numbers.

